

MEMORIA DEL PROYECTO

Sistema IoT de monitorización y control de red de agua
doméstica con detección de fugas

Alejandro Moscardó Vilar y Miguel Pérez Pomares

Internet de las Cosas | Pietro Manzoni

Tabla de contenido

1. Contexto y objetivos del proyecto	2
Contexto	2
Objetivos.....	2
Alcance del prototipo.....	2
2. Trabajo realizado y metodología seguida	3
Metodología de trabajo	3
Pruebas iniciales con el ESP32	3
Integración del relé y control de la electroválvula.....	3
Lectura del sensor de caudal.....	4
Montaje del circuito hidráulico	4
Desarrollo del firmware del ESP32	5
Comunicación MQTT, procesamiento y dashboard web	5
Validación del funcionamiento	6
3. Resultados obtenidos	6
Resultado general del prototipo	6
Funcionamiento del microcontrolador	7
Backend en Node-RED	8
Dashboard web	8
Evaluación de los resultados.....	9
4. Problemas encontrados y soluciones aplicadas	9
Fugas y conexiones del circuito hidráulico	9
Influencia de la presión en la medida de caudal	10
Desincronización del estado de la electroválvula.....	10
Pulsos espurios al conmutar la electroválvula	11
Posibles estados retenidos en MQTT.....	11
Problemas al incorporar Bluetooth	11
Decisiones aplicadas.....	12
5. Conclusiones.....	12

1. Contexto y objetivos del proyecto

Contexto

El proyecto nace a partir de una necesidad práctica en el ámbito doméstico: disponer de un sistema que permita supervisar el comportamiento de una red de agua, detectar situaciones anómalas y actuar de forma remota sobre sus elementos principales. En instalaciones de riego o distribución de agua en una vivienda, una fuga o un consumo no previsto puede pasar desapercibido durante bastante tiempo, con el consiguiente desperdicio de agua y el riesgo de daños materiales.

Para abordar este problema se plantea el desarrollo de un prototipo IoT basado en un microcontrolador ESP32, capaz de medir el caudal de agua mediante un sensor de flujo y controlar una electroválvula mediante un módulo relé. El sistema se apoya en conectividad inalámbrica y en una arquitectura de comunicación orientada a mensajes, de forma que los datos puedan ser procesados y visualizados desde una interfaz web accesible desde dispositivos de la red local. La solución propuesta combina hardware, firmware y software de supervisión. El ESP32 recoge las medidas del sensor, publica información del estado del sistema y recibe órdenes de control. Un broker MQTT, junto con herramientas como Node-RED, permite procesar los mensajes, generar alertas y servir la información al panel web. Este enfoque facilita separar la adquisición de datos, la lógica de control y la visualización, haciendo que el sistema sea más modular y ampliable.

Objetivos

El objetivo principal del proyecto es diseñar e implementar un prototipo funcional de monitorización y control de una red de agua doméstica, capaz de medir el caudal en tiempo real, detectar posibles fugas y permitir la apertura o cierre remoto de una electroválvula.

Como objetivos específicos, el proyecto busca integrar el sensor de caudal YF-S201 con el ESP32, desarrollar el firmware necesario para el cálculo del caudal y la comunicación con el sistema central, configurar una arquitectura basada en MQTT, implementar un backend ligero para el tratamiento de los datos y crear una interfaz web que muestre el estado del sistema, las alertas y las gráficas de consumo.

Además, se pretende que el prototipo sirva como base para futuras ampliaciones, como la incorporación de una bomba de presión, el uso de almacenamiento histórico más avanzado, la mejora de los algoritmos de detección de anomalías o la integración con otros sistemas domóticos.

Alcance del prototipo

El alcance de esta entrega se centra en la construcción de un prototipo representativo de una instalación doméstica. El sistema no pretende sustituir directamente a una instalación profesional certificada, sino demostrar la viabilidad técnica de una solución IoT capaz de medir, comunicar, visualizar y actuar sobre una red hidráulica sencilla.

La instalación física se simula mediante un circuito de agua compuesto por depósito, sensor de caudal, electroválvula y salida. A nivel electrónico, el ESP32 actúa como controlador principal y se encarga tanto de la lectura del sensor como del accionamiento del relé. A nivel software, la comunicación y visualización de datos permiten comprobar el funcionamiento global del sistema y validar su utilidad en un entorno doméstico.

2. Trabajo realizado y metodología seguida

Metodología de trabajo

El desarrollo del proyecto se realizó siguiendo una metodología incremental, basada en dividir el sistema en fases pequeñas y validar cada una antes de pasar a la siguiente. Esta forma de trabajo resultó especialmente adecuada porque el proyecto combina componentes hidráulicos, electrónicos y software, y un error en cualquiera de estas partes podía dificultar la localización del problema si se integraba todo desde el principio.

Por ello, primero se comprobaron los elementos básicos del ESP32 y después se fueron incorporando progresivamente los actuadores, los sensores, la comunicación inalámbrica, el procesamiento de datos y la interfaz web. Cada fase se probó de forma aislada mediante pruebas sencillas y, una vez validada, se integró con la fase anterior. El orden seguido fue coherente con la planificación del proyecto: pruebas iniciales del microcontrolador, control del relé, activación de la electroválvula, lectura del sensor de caudal, conexión WiFi, comunicación MQTT, flujo de procesamiento en Node-RED y desarrollo del dashboard web.

Pruebas iniciales con el ESP32

El primer paso consistió en realizar pruebas básicas con el ESP32 para comprobar que el entorno de desarrollo, la placa y la carga de programas funcionaban correctamente. Para ello se utilizaron pequeños montajes con LEDs, que permitieron verificar la configuración de pines digitales, la compilación desde Arduino IDE y la subida del firmware al microcontrolador.

Estas pruebas iniciales sirvieron para confirmar que el ESP32 respondía correctamente a señales digitales HIGH y LOW. Además, permitieron familiarizarse con el flujo de trabajo de programación, carga y comprobación mediante el monitor serie antes de conectar componentes de mayor consumo o elementos externos como el relé y la electroválvula.

Integración del relé y control de la electroválvula

Una vez comprobado el funcionamiento básico del ESP32, se pasó a probar el módulo relé. El módulo utilizado dispone de dos canales, de forma que el primer canal se empleó para controlar la electroválvula y el segundo quedó reservado para una posible ampliación futura con una bomba de presión. Para la prueba del canal principal se conectó la alimentación lógica del relé al ESP32, utilizando inicialmente 3.3V en DC+ y GND en DC-, y se conectó la entrada IN1 al GPIO 25. Con esta conexión se enviaron señales HIGH y LOW desde el ESP32 para comprobar que el relé conmutaba correctamente. Esta prueba permitió verificar el

comportamiento del módulo antes de introducir la alimentación de 12V de la electroválvula. También se comprobó qué nivel eléctrico activaba el canal del relé, ya que en el montaje final el firmware mantiene separado el estado lógico de la válvula, abierta o cerrada, del nivel aplicado al pin de control.

Después se integró la electroválvula de 12V normalmente cerrada. Como la fuente de alimentación incorporaba un conector cilíndrico, fue necesario cortar el cable para separar el conductor positivo del negativo. El conductor negativo de la fuente se conectó directamente a una de las patillas de la electroválvula. El conductor positivo de 12V se llevó a la entrada COM1 del relé y, desde la salida NO1, se conectó la línea positiva conmutada a la otra patilla de la electroválvula.

Con este montaje el ESP32 no alimenta directamente la electroválvula, sino que únicamente controla el relé. La alimentación de 12V queda aislada del microcontrolador y solo pasa hacia la electroválvula cuando el relé cierra el circuito. Al enviar órdenes de apertura y cierre desde el ESP32 se comprobó que la electroválvula abría o cerraba correctamente. La validación se realizó de forma sencilla soplando a través de la válvula para comprobar si el paso de aire quedaba permitido o bloqueado según el estado del relé.

Lectura del sensor de caudal

Tras validar el actuador principal, se incorporó el sensor de caudal YF-S201. Este sensor proporciona una señal digital por pulsos proporcional al flujo de agua que atraviesa su rotor interno. Para conectarlo al ESP32 se llevó el cable amarillo de señal al GPIO 27, el cable negro a GND y el cable rojo a la alimentación de 5V. Además, se colocó una resistencia pull-up de 10 k Ω entre la línea de señal y 3.3V para estabilizar la lectura digital.

La prueba del sensor se realizó soplando en el sentido correcto de paso del caudal. Al hacerlo, el rotor interno generaba pulsos y el ESP32 podía detectar actividad en el pin de entrada. Esta prueba permitió confirmar tanto el cableado como el sentido de instalación del sensor, aspecto importante porque el YF-S201 debe montarse respetando la dirección de flujo indicada por el fabricante.

Montaje del circuito hidráulico

Una vez comprobados el sensor y la electroválvula por separado, se montó el circuito hidráulico del prototipo. Para simular un depósito de agua se utilizó una botella, a cuyo tapón se conectó un tubo de media pulgada. Esta salida del depósito se llevó directamente a la entrada del sensor de caudal, de forma que todo el agua que saliera del depósito atravesara primero el sensor.

Desde la salida del sensor de caudal se conectó otro tramo de tubo hasta la entrada de la electroválvula. Finalmente, desde la salida de la electroválvula se añadió un último tubo dirigido a un recipiente de recogida, con el objetivo de comprobar el funcionamiento del sistema sin desperdiciar agua. El orden físico del circuito quedó, por tanto, como depósito, sensor de caudal, electroválvula y recipiente de salida.

Este montaje permitió validar el comportamiento completo del sistema hidráulico: el sensor quedaba situado antes de la electroválvula para medir el flujo generado desde el depósito, mientras que la electroválvula actuaba como elemento de corte del paso de agua antes de la salida final. Así se pudo comprobar de forma

práctica si las órdenes enviadas desde el ESP32 modificaban realmente el paso de agua en el circuito.

Desarrollo del firmware del ESP32

Después de comprobar por separado los elementos físicos, se abordó la implementación del software embebido. El firmware se desarrolló con Arduino IDE y se estructuró desde el inicio para separar las responsabilidades principales: conexión a la red WiFi, comunicación MQTT, lectura del sensor de caudal, control de la electroválvula y publicación del estado del sistema.

Para ello se utilizaron las librerías WiFi.h, PubSubClient.h y ArduinoJson.h. La librería WiFi.h permite conectar el ESP32 a la red inalámbrica; PubSubClient.h proporciona el cliente MQTT necesario para publicar datos y recibir comandos; y ArduinoJson.h facilita la construcción de mensajes estructurados en formato JSON, especialmente para enviar medidas de caudal, número de pulsos y estado del sistema.

En el firmware se definieron los pines principales del prototipo: GPIO 25 para el relé de la electroválvula y GPIO 27 para la señal del sensor de caudal. La electroválvula se controla mediante una función específica que actualiza el estado lógico de la válvula y aplica al pin el nivel necesario para abrirla o cerrarla. Esta separación evita confusiones con el comportamiento eléctrico del relé, que en el prototipo actúa como relé activo en bajo.

La lectura del caudal se implementó mediante una interrupción asociada al pin del sensor. Cada flanco detectado incrementa un contador de pulsos y, de forma periódica, el programa calcula el caudal aproximado en litros por minuto a partir de esos pulsos. Posteriormente, el ESP32 publica la medida en el topic MQTT casa/agua/caudal usando un mensaje JSON con el caudal, los pulsos registrados y una marca temporal interna.

También se implementó la recepción de comandos remotos. El ESP32 se suscribe al topic casa/agua/valvula y, cuando recibe un mensaje ON u OFF, abre o cierra la electroválvula. Tras cada cambio publica el estado actualizado en casa/agua/estado. De este modo, el resto del sistema puede conocer el estado real de la válvula y reflejarlo en la interfaz web.

Comunicación MQTT, procesamiento y dashboard web

La comunicación del sistema se diseñó alrededor de MQTT para desacoplar el firmware, el procesamiento de datos y la visualización. Durante el desarrollo se empleó HiveMQ como broker MQTT compatible con los topics definidos para el proyecto, manteniendo la arquitectura preparada para poder sustituirlo por un broker local como Mosquitto. El ESP32 publica datos de caudal, estado y alertas, mientras que el dashboard y los flujos de procesamiento se suscriben a esos topics para actualizar la información en tiempo real.

En Node-RED se preparó un flujo para recibir los mensajes de caudal, calcular consumo acumulado diario y semanal, procesar alertas y publicar información ya tratada en un topic específico de consumo procesado. También se incluyó la estructura para publicar comandos de control hacia la válvula y se dejó deshabilitado el control de bomba, ya que dicho componente no forma parte del prototipo físico actual.

El dashboard web se desarrolló como una interfaz responsive en HTML, CSS y JavaScript. La interfaz se conecta al broker MQTT mediante WebSockets, recibe los datos publicados por el ESP32 y actualiza en tiempo real el caudal, el número de pulsos, el estado de la electroválvula, los avisos de fuga y las estimaciones de consumo. Para la representación gráfica del consumo se utilizó Chart.js, permitiendo alternar entre vistas diarias y semanales.

Validación del funcionamiento

La validación del sistema se realizó de forma progresiva. Primero se comprobó que el ESP32 podía activar salidas digitales mediante LEDs. Después se verificó la conmutación del relé y, posteriormente, la apertura y cierre de la electroválvula con alimentación externa de 12V. Más adelante se comprobó la recepción de pulsos del sensor de caudal y, finalmente, se integraron las comunicaciones WiFi y MQTT para observar los datos desde el software.

Una vez integrado el conjunto, se implementaron reglas simples de detección de anomalías. El sistema genera una alerta inmediata si el caudal supera 0,1 L/min mientras la válvula está cerrada, ya que esta situación puede indicar una fuga o un cierre incorrecto. También genera una alerta si se mantiene un caudal superior a 0,1 L/min durante 60 segundos consecutivos; en ese caso ordena el cierre automático de la electroválvula como medida de seguridad. Con estas pruebas se obtuvo un prototipo funcional capaz de medir, comunicar, visualizar y actuar sobre una instalación hidráulica sencilla.

3. Resultados obtenidos

Resultado general del prototipo

El resultado final del proyecto fue un prototipo funcional de sistema IoT para monitorización y control de una red de agua doméstica a pequeña escala. El sistema consiguió integrar el circuito hidráulico, el control electrónico mediante ESP32, la comunicación MQTT, un backend sencillo en Node-RED y un dashboard web para visualizar el estado del sistema.

Aunque se trata de un montaje de laboratorio y no de una instalación definitiva, el prototipo permitió demostrar el flujo completo de funcionamiento previsto: el agua circula desde el depósito, atraviesa el sensor de caudal, pasa por la electroválvula y llega al recipiente final; al mismo tiempo, el ESP32 registra pulsos del sensor, controla la apertura o cierre de la válvula y comunica los datos al resto del sistema.



Figura 1. Montaje hidráulico del prototipo.

Funcionamiento del microcontrolador

El microcontrolador ESP32 funcionó correctamente como núcleo del sistema. Se consiguió controlar la electroválvula mediante el relé conectado al GPIO 25 y se pudo leer la señal del sensor de caudal conectado al GPIO 27. El firmware fue capaz de contar pulsos, calcular una estimación de caudal en litros por minuto y publicar esa información mediante MQTT.

También se validó la recepción de comandos remotos para abrir y cerrar la electroválvula. Cuando el sistema recibe una orden en el topic correspondiente, actualiza el estado de la válvula y publica de nuevo el estado general. Esto permitió comprobar que el ESP32 no solo actúa como dispositivo de medición, sino también como actuador controlable desde el sistema software.

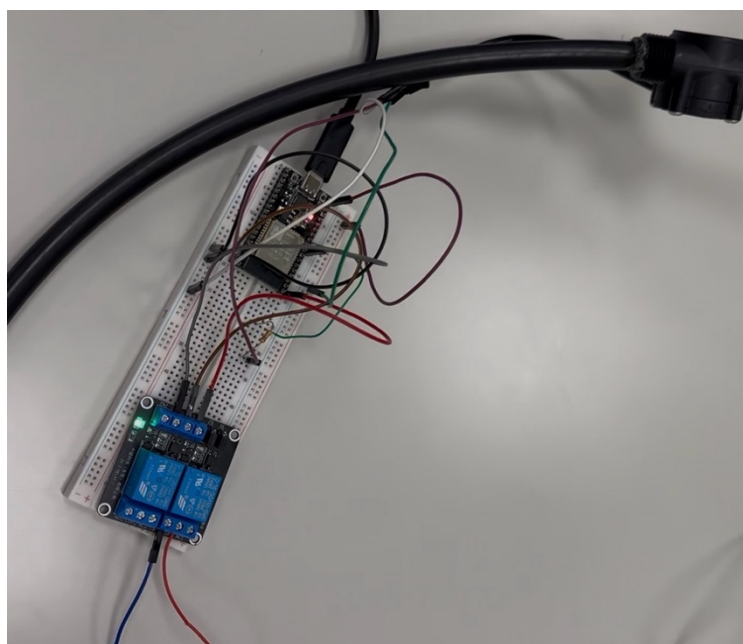


Figura 2. Montaje electrónico del sistema de control.

Backend en Node-RED

Se obtuvo un backend sencillo pero funcional en Node-RED. Este backend se encarga de suscribirse a los topics MQTT del sistema, recibir los mensajes enviados por el ESP32, procesar la información de caudal y generar datos de consumo acumulado. También procesa las alertas publicadas por el microcontrolador para convertirlas en mensajes más interpretables dentro del sistema.

El flujo de Node-RED sirvió como capa intermedia entre el dispositivo físico y la interfaz web. Gracias a esta capa se pudo mantener el firmware más centrado en la adquisición y control, dejando el cálculo de consumo y el tratamiento de mensajes para el backend. El resultado fue una arquitectura más ordenada y fácil de ampliar.

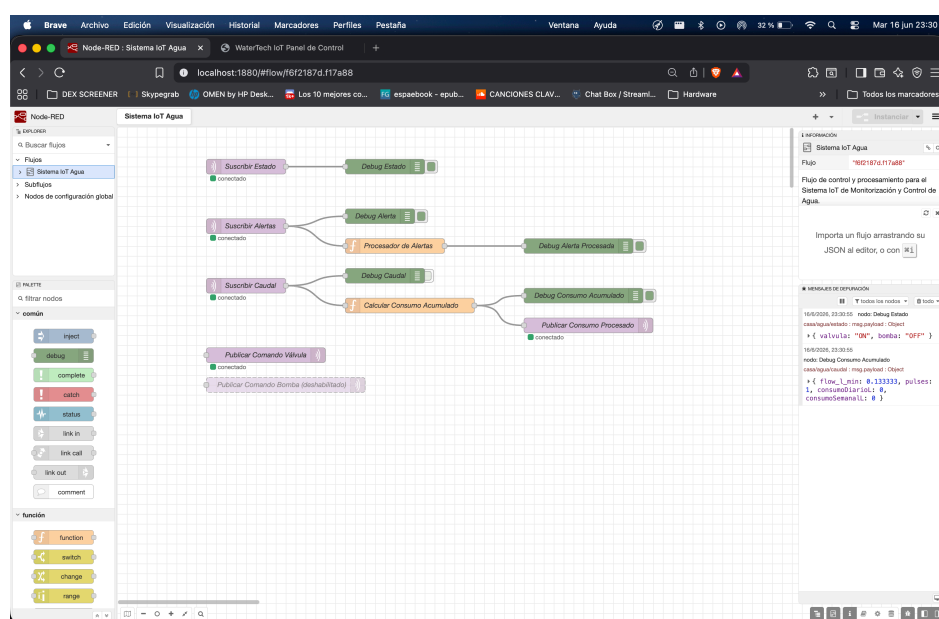


Figura 3. Backend de procesamiento implementado en Node-RED.

Dashboard web

El dashboard web quedó operativo y mostró resultados en tiempo real a partir de los mensajes MQTT. La interfaz permite visualizar el caudal actual, los pulsos recibidos del sensor, el estado de la electroválvula, las alertas y las estimaciones de consumo diario y semanal. Además, incorpora gráficas para representar la evolución del consumo de forma más clara.

Desde el panel también se puede enviar la orden de apertura o cierre de la electroválvula. Esto permitió validar el control remoto del sistema desde una interfaz accesible para el usuario, cerrando el ciclo completo entre medición, comunicación, visualización y actuación.

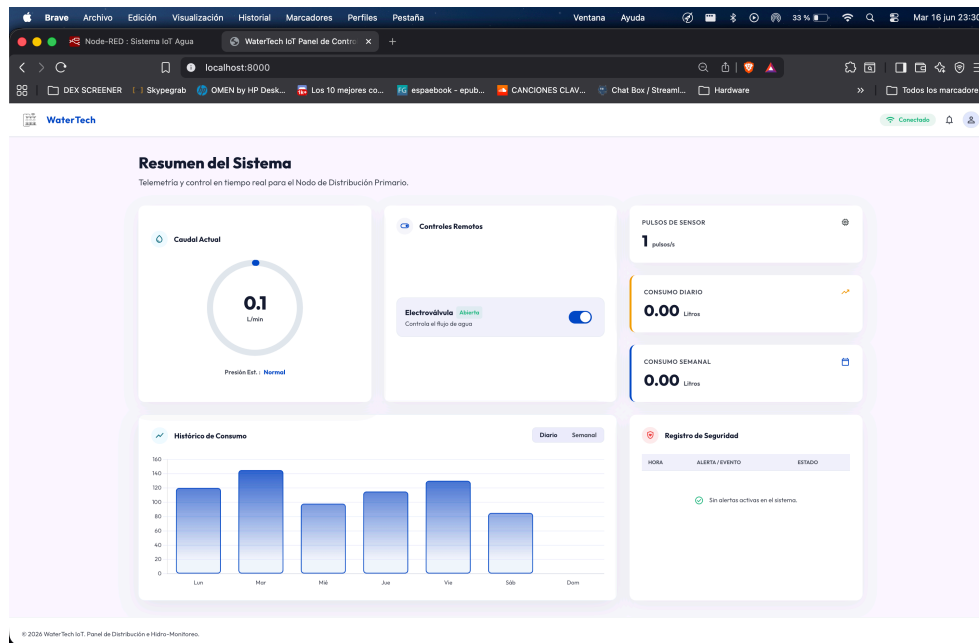


Figura 4. Dashboard web con visualización y control del sistema.

Evaluación de los resultados

Los resultados obtenidos fueron positivos porque se consiguió que las partes principales del proyecto trabajasen de forma integrada. El sistema no se quedó únicamente en una prueba aislada de hardware o en una interfaz simulada, sino que conectó el prototipo físico con una cadena software completa basada en MQTT, Node-RED y un dashboard web.

Como resultado, el proyecto cumple los objetivos principales planteados: monitorizar el flujo de agua, controlar una electroválvula, transmitir datos mediante MQTT, procesar información en un backend ligero y visualizar el estado del sistema desde una interfaz web. Además, se implementó una lógica básica de detección de fugas, lo que añade una primera capa de automatización y seguridad al prototipo.

El sistema queda preparado como una base funcional sobre la que se podrían realizar mejoras futuras, como sustituir el broker público por un broker local estable, añadir una bomba de presión, almacenar históricos de forma persistente o mejorar los algoritmos de detección de anomalías.

4. Problemas encontrados y soluciones aplicadas

Fugas y conexiones del circuito hidráulico

La mayor parte de los problemas encontrados durante el desarrollo estuvieron relacionados con el montaje físico del circuito hidráulico. Aunque el esquema general era sencillo, en la práctica fue necesario adaptar los tubos, uniones y conexiones para que el agua circulara correctamente desde el depósito hasta el recipiente final sin pérdidas.

El primer problema fue que algunos tubos de media pulgada no encajaban bien con las conexiones disponibles del sensor de caudal, la electroválvula y el tapón

de la botella utilizada como depósito. Esto provocaba uniones poco firmes y dificultaba mantener el circuito estable durante las pruebas con agua.

Después de conseguir montar el circuito completo aparecieron fugas en varios puntos de unión. Estas fugas impedían validar correctamente el comportamiento del sistema, ya que parte del agua se escapaba antes de atravesar todo el recorrido previsto o antes de llegar al recipiente de salida.

La primera solución aplicada fue reforzar las uniones con una pistola de silicona caliente y adaptar la forma de algunos tubos aplicando calor con un mechero. Al calentar ligeramente los extremos de los tubos fue posible moldearlos para que encajaran mejor en las conexiones disponibles. Sin embargo, esta solución no fue suficiente para eliminar completamente las fugas de agua.

Debido a que las pérdidas de agua persistían, fue necesario comprar piezas adicionales en Leroy Merlin para rehacer las uniones de forma más fiable. En total se adquirieron ocho piezas: cuatro adaptadores roscados con espiga para tubo de 10 mm y cuatro uniones roscadas hembra-hembra. Aunque en el momento de la compra se identificaron de forma práctica como roscas con pitorro y roscas hembra-hembra, su función fue permitir una conexión más estanca y compatible entre el tubo flexible, el sensor de caudal y la electroválvula.

Además, se incorporó cinta de teflón en las roscas para mejorar el sellado de las conexiones y reducir las fugas. Con esta modificación el montaje hidráulico quedó más estable y permitió continuar con las pruebas del sistema sin que las pérdidas de agua alterasen de forma significativa la validación del caudal, la electroválvula y las alertas de fuga.

Esta solución no se considera una solución definitiva para una instalación real, pero sí fue adecuada para un prototipo académico de laboratorio. Permitted estabilizar el circuito hidráulico lo suficiente para comprobar el funcionamiento del sensor de caudal, la electroválvula y el control desde el ESP32.

Influencia de la presión en la medida de caudal

Durante las pruebas también se detectó un inconveniente relacionado con la precisión del sensor de caudal. La medida obtenida podía variar en función de la presión ejercida sobre el circuito. Si la presión se mantiene constante y el sensor se calibra para esas condiciones, la estimación del caudal resulta válida; sin embargo, al modificar la altura del depósito, o al introducir una bomba que aumente o reduzca la presión, el caudal registrado puede dejar de corresponderse con el caudal real.

Como posible mejora, se plantea incorporar un sensor de presión antes del sensor de flujo. De esta forma el sistema podría conocer las condiciones hidráulicas de entrada y aplicar una corrección o calibración dinámica sobre la lectura del caudalímetro, obteniendo medidas más fiables aunque varíe la presión del depósito o del sistema de impulsión.

Desincronización del estado de la electroválvula

Durante la integración final se detectó una desincronización entre el estado físico de la electroválvula y el mostrado en el dashboard. La causa fue la diferente interpretación de los valores ON y OFF respecto al nivel eléctrico del pin y a la polaridad del relé. Para resolverlo se separaron explícitamente en el firmware y en

la interfaz el estado físico de la válvula, el nivel aplicado al pin y el comando MQTT correspondiente, evitando que una válvula alimentada y abierta apareciese como cerrada.

Pulsos espurios al conmutar la electroválvula

También se observaron pulsos espurios del caudalímetro al abrir o cerrar la electroválvula. La conmutación del relé generaba transitorios eléctricos que podían interpretarse como flujo real y aumentar incorrectamente el consumo acumulado. Se estabilizó la entrada mediante INPUT_PULLUP, se añadió un tiempo mínimo de 1000 microsegundos entre pulsos válidos y se descartaron las lecturas durante 1500 milisegundos después de cada conmutación, borrando además el contador acumulado en ese instante.

Posibles estados retenidos en MQTT

Respecto a la posible persistencia de un estado antiguo, se comprobó que Node-RED únicamente publicaba con retención el consumo procesado y no el estado de la válvula. La desincronización podía proceder de un mensaje retenido previamente en el broker público o de otro cliente que utilizase los mismos topics genéricos. Por ello el dashboard se configuró para ignorar mensajes retenidos en casa/agua/estado y se identificó como mejora futura el uso de un prefijo MQTT exclusivo para el proyecto.

Problemas al incorporar Bluetooth

Otro problema relevante apareció al intentar incorporar control local mediante Bluetooth clásico en el ESP32. La idea inicial era añadir una vía alternativa de control local, además de MQTT, usando la librería BluetoothSerial.h y comandos de texto para abrir o cerrar la electroválvula, consultar el estado del sistema y enviar alertas.

Al añadir Bluetooth al firmware, el tamaño del sketch aumentó considerablemente. Con la configuración inicial de placa, el programa superaba el espacio disponible para la aplicación, ya que el binario combinaba WiFi, MQTT, ArduinoJson y Bluetooth. Para resolver este primer bloqueo se cambió la placa seleccionada en Arduino IDE a ESP32 Dev Module y se configuró el esquema de particiones Huge APP (3MB No OTA/1MB SPIFFS). Con este cambio el firmware volvió a compilar correctamente.

Sin embargo, aunque el problema de compilación quedó resuelto, no se consiguió subir correctamente el firmware al ESP32. Durante la carga aparecieron errores relacionados con el puerto serie /dev/cu.usbserial-0001, primero indicando que el puerto estaba ocupado o no existía y después mostrando errores de comunicación durante la conexión con el dispositivo.

Durante el diagnóstico se consideraron varias causas posibles: el puerto podía estar ocupado por el monitor serie, el ESP32 podía no estar entrando correctamente en modo bootloader, el cable USB o la conexión podían no ser estables, algún periférico conectado podía interferir durante la subida o la configuración de placa podía no ajustarse completamente a la placa física utilizada.

Se probaron distintas medidas de corrección, como cerrar el monitor serie, reconectar la placa, reducir la velocidad de subida, usar la secuencia manual con el botón BOOT, revisar el puerto seleccionado y plantear la desconexión temporal de periféricos durante la carga. A pesar de ello, no se consiguió completar la subida del firmware con Bluetooth incorporado.

Decisiones aplicadas

Para evitar que este problema bloqueara el avance del proyecto, se decidió retirar la funcionalidad Bluetooth de la versión final del prototipo. Esta decisión permitió mantener el sistema estable y centrar el desarrollo en la comunicación WiFi mediante MQTT, que ya cumplía los objetivos principales de monitorización, control remoto y visualización desde el dashboard web.

La funcionalidad Bluetooth queda, por tanto, como una posible mejora futura. En una versión posterior se podría retomar el diagnóstico usando una placa con mayor margen de memoria, una configuración de particiones validada desde el inicio, pruebas de carga sin periféricos conectados y una revisión completa del cable USB, drivers y modo bootloader.

En conjunto, los problemas encontrados permitieron ajustar el alcance del prototipo y reforzar la metodología seguida: validar cada parte por separado, resolver primero la estabilidad física del circuito y priorizar una arquitectura software funcional antes de incorporar funcionalidades secundarias.

5. Conclusiones

El desarrollo del proyecto puede considerarse satisfactorio, ya que se ha conseguido construir un prototipo funcional que cumple los objetivos principales planteados al inicio. El sistema permite monitorizar el flujo de agua, controlar una electroválvula, comunicar los datos mediante MQTT, procesarlos con Node-RED y mostrarlos en un dashboard web accesible para el usuario.

Uno de los aspectos más positivos del resultado obtenido es que el prototipo no se limita a una simulación software, sino que integra hardware real, un circuito hidráulico físico y una cadena completa de comunicación y visualización. Esto demuestra que la solución propuesta es viable como base para una futura integración en sistemas reales de riego o distribución de agua doméstica.

Durante el proyecto se comprobó la importancia de avanzar por fases y validar cada componente antes de integrarlo con el resto. Las pruebas iniciales con el ESP32, la validación del relé, la conexión de la electroválvula, la lectura del sensor de caudal y la integración progresiva con MQTT y el dashboard permitieron reducir la complejidad del desarrollo y localizar mejor los problemas.

También se han identificado limitaciones propias de un prototipo. El montaje hidráulico requirió ajustes manuales para evitar fugas, la precisión del caudal depende de las condiciones de presión y la funcionalidad Bluetooth tuvo que descartarse para mantener la estabilidad del sistema. Además, el dashboard representa el estado lógico publicado por el ESP32, pero no dispone de un sensor que confirme la posición física de la electroválvula. Una versión más robusta podría incorporar medida de corriente, además del sensor de presión planteado, para detectar cualquier discrepancia entre la orden enviada y el estado real del

circuito. Aun así, estas dificultades no impidieron alcanzar el objetivo principal y aportaron aprendizaje útil sobre las restricciones reales de trabajar con hardware, comunicaciones y montaje físico.

En conclusión, estamos satisfechos con el resultado obtenido. El proyecto ha permitido demostrar que es posible desarrollar un sistema IoT de bajo coste para supervisar y controlar una red de agua, con capacidad de detección básica de fugas y control remoto. Además, la arquitectura empleada deja el sistema preparado para futuras mejoras, como una instalación más robusta, almacenamiento histórico persistente, uso de un broker local, incorporación de nuevos sensores o integración con sistemas domóticos reales.